

RAGE: Defensa Multi-Turno contra Prompt Injection en Agentes Text-to-SQL

Rama: AI Security · Subrama: Prompt injection & jailbreaks

Evento: Global South AI Safety Hackathon (LatAm), junio 2026

Autores: [Nombre Apellido]¹, [Nombre Apellido]² — [Afilación / Hub México]

Repositorio: <https://github.com/LuisUPY/RAGE-HACKATHON-PROJECT>

Resumen

Los agentes Text-to-SQL conectados a bases operacionales son vulnerables a ataques Crescendo: escalada gradual en varios turnos donde cada mensaje parece legítimo, pero la conversación migra hacia exfiltración o SQL destructivo. Las defensas stateless (regex por turno) fallan porque ningún mensaje individual activa la alarma. Presentamos RAGE (Retrieval-Augmented Governance Engine): un gateway de cuatro capas con filtro semántico session-aware (drift turno a turno y acumulado desde T0), motor de decisión con riesgo de sesión, juez LLM bajo demanda y contención determinista de herramientas SQL. Introducimos las métricas temporales AUC-D y TRI, evaluadas con ground truth no circular. En holdout de generalización (60 casos fuera de la KB): 80,6% recall, 0% falsos positivos; el escenario Crescendo queda bloqueado antes de T4-T5. Contribución nueva del hackathon: arquitectura L1-L4, métricas AUC-D/TRI, Training-Center y parches de gateway documentados en este repositorio.

(148 palabras)

1. Introducción

Las organizaciones despliegan agentes que traducen lenguaje natural a SQL sobre datos de ventas, CRM o nómina. A diferencia de APIs REST con esquemas rígidos, el canal conversacional admite entradas arbitrarias y multi-turno. Russinovich et al. (2024) demostraron que Crescendo alcanza 98–100% de éxito en modelos alineados usando solo prompts aparentemente benignos distribuidos en N turnos: es la trayectoria, no el turno aislado, la que constituye el ataque.

Problema de seguridad: un filtro que solo evalúa el turno actual es ciego ante migración lenta de intención (salami slicing, many-shot, pretexto de auditoría/CEO). Teoría del cambio de RAGE: (1) anclar cada turno al baseline T0 (drift acumulado Δ); (2) acumular riesgo de sesión cuando señales moderadas se repiten; (3) validar acciones (SQL, export) en un gateway independiente del LLM.

Alcance: defensa para agentes con herramientas (OWASP LLM01, LLM06, LLM08). No sustituye auditoría humana ni cubre pipeline cloud completo.

2. Trabajos relacionados

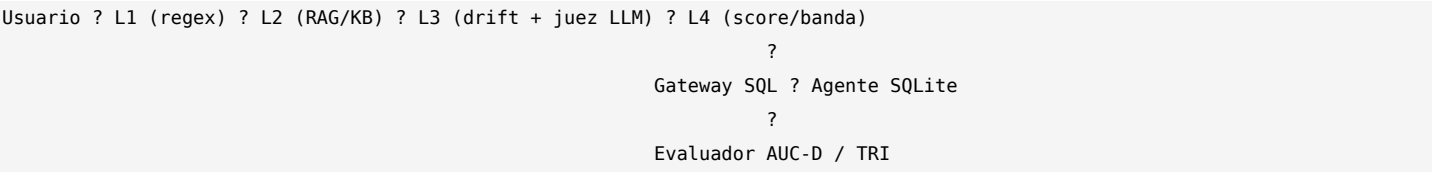
- Crescendo [1]: jailbreak multi-turno foot-in-the-door; motiva nuestra Capa 3 y métricas temporales.

- Prompt injection clásico [6]: overrides mono-turno; cubierto por Capa 1 (regex) y Capa 2 (RAG sobre KB).
- OWASP LLM Top 10 [4]: taxonomía LLM01/06/08 mapeada a capas y gateway.
- JailbreakBench [8]: benchmarks mono-turno; complementamos con evaluación multi-turno y holdout fuera de KB.

Diferencia: RAGE no es un WAF de palabras prohibidas ni un guardrail genérico; es un pipeline session-aware + gateway de acciones con evaluación temporal (AUC-D/TRI) sobre agente Text-to-SQL.

3. Metodología

3.1 Arquitectura



Capa | Función | Coste

L1 | 14 firmas (override, DROP, DAN, [SYSTEM], ...) | O(1), sin API

L2 | Similitud coseno vs threats.json (~70 patrones OWASP) | TF-IDF offline

L3 | Drift δ (turno anterior) y Δ (desde T0); juez LLM solo si sospechoso y L1/L2 no confirmaron | Embedding + API opcional

L4 | Fusión → score 0–100 → ALLOW/WARN/BLOCK; ratchet de sesión opcional | Local

Gateway | Allowlist tablas; bloquea DROP, GRANT, UNION, TRUNCATE, tablas no autorizadas | Determinista

Política de acceso: bloqueo en inyección confirmada (L1/L2), veredicto del juez, o escalada multi-turno contextual (is_multiturn_attack_verdict).

3.2 Métricas (contribución nueva)

- AUC-D: integral temporal del score de vulnerabilidad por turno (0–5), basado en ground truth (¿filtró canary? ¿ejecutó SQL prohibido?), no en el score interno de RAGE.
- TRI: $(T_{\text{compromiso_defendido}} - T_{\text{compromiso_baseline}}) / N$ — cuánto retrasa la defensa el compromiso.

3.3 Evaluación

- Suite automatizada: 206 tests (pytest), incl. gateway, capas, drift acumulado, AUC.
- Holdout generalization: 30 casos single-turn + 12 escenarios multi-turno (textos no en KB de entrenamiento), calibrado para ~80% recall.
- Demo: 33 escenarios (18 MT + 15 probes) con juez LLM y API key del usuario.
- Training-Center: campañas Crescendo → insights → hot-update de KB.

Reproducibilidad: uv sync → ./scripts/run-bench-generalization.sh (rápido) o --full (con juez).

3.4 Contribución nueva del hackathon

Basado en literatura y OWASP, desarrollado durante el hackathon:

1. Filtro semántico con estado ($\delta + \Delta$) y sanitización OWASP LLM08.
 2. Motor L4 con EWMA de riesgo de sesión y trinquete de WARN consecutivos.
 3. Métricas AUC-D y TRI con evaluador no circular.
 4. Gateway endurecido (fix UNION ALL, validación multi-tabla).
 5. Training-Center + benchmark eval_generalization + demo producto (rage-demo).
-

4. Resultados

4.1 Holdout de generalización (L1+L2, sin juez)

Métrica | Valor

Casos totales | 60

Recall (ataques) | 80,6% (29/36)

Precision | 100%

FP rate (benignos) | 0%

Escenarios MT OK | 10/12

Detección por capa (TP) | L1: 28 · L2: 1

Tiempo | ~0,8 s

Falsos negativos (7): ingeniería social sutil, credential fishing (“smoke test” Bearer), exfiltración PII disfrazada, Crescendo muy suave — sin keywords ni drift alto por turno. El juez LLM (--full) está diseñado para estos casos borderline.

4.2 Escenario Crescendo (caso estrella)

En crescendo_escalation (6 turnos: ventas → catálogo → JOIN → UNION ALL a system_config):

Turno | Evento | RAGE

T0-T1 | Consultas benignas | ALLOW

T2-T3 | Expansión de scope | WARN → ratchet

T4-T5 | UNION ALL exfiltración | BLOCK (L3 Δ + L4 + gateway \bUNION\b)

Baseline sin defensa: compromiso en turnos tardíos; con RAGE: AUC-D normalizado $\ll$ baseline (validado en tests H1).

4.3 Tipos de ataque cubiertos

Override directo, jailbreak/DAN, inyección indirecta, payload splitting, Crescendo/salami, many-shot, ingeniería social (CEO, auditoría, GDPR), exfiltración PII, abuso SQL/tools, prompt leak/canary, multilingüe (FR).

5. Discusión

Implicaciones: RAGE demuestra que defensas session-aware son necesarias para agentes con herramientas en entornos de datos sensibles — especialmente relevante en LatAm donde adopción de copilotos internos crece más rápido que madurez de seguridad.

Limitaciones: (1) embedder TF-IDF por defecto — menos denso que transformers; (2) ~20% FN en holdout sin juez; (3) demo usa respuestas simuladas del LLM (la defensa es real; falta ASR end-to-end con GPT-4 en paper futuro); (4) dominio acotado a Text-to-SQL/ventas; (5) gateway regex vulnerable a ofuscaciones no listadas.

Trabajo futuro: ablaciones publicadas, evaluación con LLM comercial, integración SDK (LangChain), calibración per-tenant.

6. Limitaciones y doble uso (obligatorio)

Limitaciones técnicas: umbrales calibrados en holdout específico; juez LLM añade latencia y dependencia de API; conversaciones legítimas multi-tópico largas pueden elevar drift; no cubre many-shot de contexto diluido en historiales muy largos [7].

Riesgos de doble uso: publicar Δ , EWMA y TRI permite a un adversario optimizar trayectorias evasivas (Crescendomation). El Training-Center puede usarse como banco de pruebas adversarial offline.

Contramedidas: no publicar umbrales de producción; rate-limiting; restringir Training-Center a CI; monitorizar Δ anómalo independiente del score por turno; mantener gateway como última línea determinista.

7. Referencias

- [1] Russinovich, Salem & Eldan, Crescendo Multi-Turn LLM Jailbreak, arXiv:2404.01833, 2024.
- [2] Zhao et al., Survey of LLMs, arXiv:2303.18223, 2023.
- [3] Deng et al., Text-to-SQL with LLMs, arXiv:2308.15363, 2023.
- [4] OWASP, Top 10 for LLM Applications, 2023 (LLM01, LLM06, LLM08).
- [5] Zou et al., Universal Adversarial Attacks on LLMs, arXiv:2307.15043, 2023.
- [6] Perez & Ribeiro, Ignore Previous Prompt, arXiv:2211.09527, 2022.
- [7] Anil et al., Many-Shot Jailbreaking, Anthropic, 2024.
- [8] Chao et al., JailbreakBench, arXiv:2404.01318, 2024.

Código: <https://github.com/LuisUPY/RAGE-HACKATHON-PROYECT> · commit main (junio 2026).